

Next: Enabling Continuous UTXO Payments with Direct Guarantees

Lu Hengrun

Abstract—Continuous payments require recipients to reuse incoming value while earlier transactions remain pending. Next supports this in an unspent transaction output (UTXO) ledger through registered guarantor organizations. After verifying and recording an output and its quorum-signed certificate (TXCer), a recipient can request a successor while public execution proceeds asynchronously. If a certified source is missing, the committee atomically executes the successor and records an obligation backed by the issuer’s reserve; the obligation closes when the source executes or, after expiry, through a reserve debit, and a source arriving after compensation repays the compensating issuer. Under fixed membership and Byzantine faults we prove consumption uniqueness, coverage of liabilities including hidden certificates, and delay neutrality: for finite sets of non-conflicting payments whose sources all execute, every successful arrival and compensation order ends with the same final CAL holders and reserve balances as undelayed execution. Restricted funding revisions preserve complete block identities. On one M4 Max host (nine services, in-memory state, NoSync wallets), 100-hop chains reach their last certified receipt in a median 158.846 ms versus 65.204 s with inter-hop public-confirmation waits, and three 30,000-payment runs complete 1,750–1,887 payments/s including drain. Nine recovery runs (disk-backed committee) close 21,600 payments and repay all 216 compensations.

Index Terms—UTXO payments, payment guarantees, Byzantine fault tolerance, authorized repair.

I. INTRODUCTION

Continuous payments require incoming value to remain usable across a sequence of transfers. In a UTXO ledger, making each successor wait for its source transaction to execute publicly inserts confirmation delays between otherwise ready payments. Consider Alice paying Bob, who uses the received output to pay Carol. Bob needs evidence that supports a new payment before Alice’s transaction executes. The difficulty is not only speed: once Bob’s payment executes first, someone must stand behind the value that Alice’s transaction has not yet delivered, and the accounting must stay correct whichever transaction reaches the ledger first, or whether Alice’s ever does.

Certificate-based payment systems show that recipient-side usability can be separated from parts of public processing. FastPay and Zef support low-latency certified payments, while Sui Lutriss combines broadcast and consensus paths for different forms of state access [1]–[3]. Payment-channel protocols provide rapid transfers through channel relationships and their liquidity [4], [5]. We address a specific native-ledger execution requirement: a certified output whose source has not yet executed should support a publicly executed successor,

and responsibility for the missing source should fall on an identified party with funds set aside for it. In Next, users spend their own routed outputs and registered organizations back unresolved sources, with no prefunded channel path between counterparties.

Three coupled problems arise. First, certificates can circulate before public registration, so public gap accounting misses some existing commitments; signing capacity must cover every certified payment, including hidden ones. Second, a source may arrive after the committee has compensated its missing value. The late arrival must not pay the same value twice, to the guarantor that advanced it and to the recipient who had already spent it; otherwise delay itself would move funds. Third, resolving a source may concern a payment whose outputs have already been consumed, so any change to its recorded funding must preserve the owner authorization, output identifiers, and commitments that successors reference. A stable transaction hash alone does not specify or authorize such a change.

Next fixes payment rights while assigning unresolved sources to their direct issuers. An output certificate (TXCer) binds a payment’s outputs to an identified issuing organization. After verifying and atomically recording an output and its TXCer, the recipient can request a successor through the output’s designated consumption organization, carrying direct input certificates as ancestry evidence; owner authorization, spend state, fees, and resource limits are checked separately.

If a certified input is still absent from public creation state, the committee atomically consumes it, records a direct obligation against the issuer, and creates the child outputs, which are final even while the obligation is open. Successful execution of the source fulfills the obligation. After its deadline, public compensation closes it through an actual reserve debit that supplies the missing backing rather than crediting the child recipient again. If the source later arrives, it repays the issuer that advanced the value, in the same transition, and creates no second output for the recipient. When all sources eventually execute, delay changes temporary capital exposure while preserving the final principal allocation.

Economic closure admits an append-only formulation. Next couples a compensation with a validated funding revision in one public transition; compensation progress therefore also requires three adaptation contributions. The accounting invariants are enforced independently of the adapter output. The repair interface installs a restricted funding replacement under the existing commitments, retaining a historical representation of the actual reserve debit. Standard wallets, synchronization, and replay read original commands; the revised body is read

by repair construction, installation, and local inspection. We organize the design into three contributions with distinct responsibilities.

C1: A layered architecture for one-round continuation. We separate member certification, recipient delivery, evidence replication, and public execution. Members atomically lock inputs and reserve capacity before signing, one parallel certification round produces a TXCer, and installation and submission proceed in the background. Registered organizations use one evidence interface within and across organizations, with payment principal separate from guarantor reserves.

C2: Direct responsibility with funded, delay-neutral accounting. We define the atomic missing-input transition and its normal, compensated, and recovered closure paths; for fixed input and certificate shapes, per-request ancestry evidence does not grow with transfer depth. Under the fixed-membership Byzantine model, we prove unique consumption, that signing capacity bounds the liability of every certificate ever signed including hidden ones, and asset conservation. We also prove that, for a finite set of non-conflicting payments whose sources all execute, any interleaving of arrivals and compensations ends with the same CAL holdings and reserve balances as undelayed execution.

C3: Identity-preserving authorized representation updates. We restrict funding changes to publicly authorized slots while preserving owner authorizations, output references, and the complete block identity, including its part-set header. Same-block batches share final-part adaptation while applying economic steps in order, and installation advances directly to a committed revision without repeating economic effects. This mechanism gives repair constructors and installers a commitment-compatible historical representation with bounded, shared adaptation work.

On a single M4 Max host with nine services, in-memory service state, and NoSync wallets, 100-hop chains reach their last certified receipt in a three-run median of 158.846 ms, versus 65.204 s with public-confirmation waits between hops. Three 30,000-payment runs complete 1,750–1,887 payments/s including background drain. A separate nine-run recovery matrix closes 21,600 payments and repays all 216 compensations; paired executions preserve the final distribution of CAL. Sections III and IV define the design; Sections V and VI develop its safety argument and evaluation.

II. BACKGROUND AND RELATED WORK

Next connects certificate-based continuation, reserve-backed public execution, and authorized historical repair. The closest work establishes precedents for certification, collateral, and controlled editing. We position Next by the responsibility assigned to a missing source and by the payment and block identities preserved when that responsibility is resolved.

A. Certificate-Based Payments and Execution

FastPay uses Byzantine consistent broadcast for low-latency payments and supports a primary-ledger deployment backed by prepaid funds [1]. Its redeemability argument permits confirmation of a valid certificate before all corresponding

incoming transfers are locally confirmed, provided the sender’s missing earlier outgoing certificates are supplied. Redemption provides a certificate-based route to the primary funding pool. Zef combines disposable accounts with private, transferable coins. Normal spending uses current coin certificates and consumption evidence rather than the complete ancestor transaction history [2].

Sui Lutris combines broadcast execution for owned objects with consensus for shared-object interactions [3]. Stingray supports concurrent transactions through Byzantine bounded counters and FastUnlock [6]; overlapping capacities and reclamation are close precedents for Next’s resource accounting. Its commutative workloads address contention differently from a payment chain in which each transaction consumes its predecessor’s output. Next focuses on publicly executing a certified UTXO consumption before the source transaction executes, while registering a funded obligation against the direct issuer. Full-output reservation and cumulative loss retention account for both publicly registered liabilities and commitments represented by hidden certificates.

B. Channels and Collateralized Guarantees

Blitz uses pay-or-revoke to replace two-phase commitment in channel payments, while Donner develops virtual channels that address the Domino attack [4], [5]. CryptoMaze provides atomic multipath channel payments with unlinkable partial payments while avoiding duplicate off-chain contracts on shared edges [7]. These mechanisms operate through channel arrangements. Next uses a native ledger with registered consumption routes and organization reserves, so counterparties need neither bilateral channels nor a prefunded channel route between them.

Snappy protects merchants accepting fast on-chain payments through collateralized approvals. Reusable payment guarantees combine privacy with customer collateral reusable across defined time windows [8], [9]. Next instead separates the user’s payment principal from additional organizational backing. Its compensation closes the missing source of an already executed child without issuing another principal credit to that child’s recipient.

HyperService uses a public Network Status Blockchain and an Insurance Smart Contract to adjudicate cross-chain execution violations and financially reverse the effects of committed transactions [10]. Next addresses a different responsibility boundary within one native UTXO ledger: compensation settles a direct issuer obligation while preserving the principal effects of already executed successors.

C. Controlled Editing and Historical Consistency

Chameleon hashes permit authorized alternative messages under the same digest and underpin editable-blockchain designs [11]. SDR studies decentralized attribute-based edit authority and dynamic delegation; Escaping moves committee selection and voting outside the consensus layer to reduce redaction waiting [12], [13]. Reconstructing Chameleon Hash

develops stronger definitions of indistinguishability and collision resistance [14]. Next uses established adaptation primitives, with public payment-state checks supplying the authority to apply a compatible replacement.

Reparo provides a publicly verifiable repair layer and can re-execute affected transactions when repairing state [15]. Next fixes payment authorizations and outputs: a public repair command accounts for compensation, and historical installation replaces the permitted funding representation without re-executing successors. Original commands and subsequent repairs remain the economic replay history.

RedactChain coordinates concurrent same-block editing conflicts and preprocessing [16]. CDEdit uses reusable privilege tokens to reduce repeated token and key generation and verification across multiple edits [17]. Next's batch retains each obligation's eligibility, reserve debit, and terminal-state check while sharing the final block representation. Its complete block identity combines the stable header hash with the original part-set header. Publicly committed logical revisions authorize exact replacement bodies; physical installation can advance directly to a later authorized revision. This separates once-only economic compensation from the physical installation of representation updates while preserving successor references and replay of the same public command history.

III. SYSTEM MODEL AND OVERVIEW

Next enables a recipient to spend a newly certified output before the transaction that creates it is publicly executed. The design separates authenticated payment rights, their public consumption, and the resolution of their funding sources. Table I summarizes the notation.

A. Participants, Assets, and Trust

Next operates a native UTXO ledger. CAL carries payment value, while FUEL pays processing fees. Users spend their own outputs; organizations provide additional CAL in public reserve accounts. Access to the fast path requires an output with a supported organization route, rather than depositing payment principal into a guarantor's reserve or opening a bilateral channel. The default fee source is the user's final FUEL outputs; an explicitly authorized organization-funded mode is also supported.

Each organization has four fixed members and a replaceable collector. Members independently validate requests and authorize payments. The collector distributes requests, assembles

certificates, and coordinates propagation without independent spending authority. A separate committee $\mathcal{C}$ has four equally weighted validators, with three votes required for commitment. It orders public commands and maintains accounts, outputs, obligations, and repair authorizations.

The adversary controls at most one member per organization and one committee validator throughout an execution. Corruption is static; users and collectors may behave arbitrarily. Honest participants share a trusted network configuration, genesis, rules, and organization registry. Each organization route identifies one active configuration and one consumption-lock domain. Owner and member signatures are unforgeable; canonical digests and original block commitments satisfy their required binding properties. Repair uses threshold adaptation, while permission to apply a repair comes from public validation and commitment. Threshold material is generated and distributed by a trusted offline dealer.

Honest signing state is atomic and neither forgotten nor rolled back. Public execution is deterministic and atomically committed; followers authenticate consecutive public prefixes. Message delivery need not be timely for safety. Progress requires sufficient responsive participants, available evidence, fees and resources, and fair delivery and scheduling. The experimental in-memory configuration provides running-state atomicity, with each run starting from fresh genesis.

B. Outputs, Routes, and Certificates

An output specifies its asset, amount, owner, and consumption route. Its identifier x is derived from the network, creating transaction, and output position. Spending is indexed by (x, i) , where ordinary outputs use instance $i = 0$. Every input of a fast payment is assigned to the same consumption organization, and its owner authorizes the new transaction. Newly created outputs may select another registered organization. Thus, cross-organization continuation changes the route of the new output without transferring the previous issuer's responsibility.

A TXCer combines a summary with a quorum certificate (QC): three distinct member signatures over the same fact c_T . The summary binds the network, transaction identity, issuer, configuration, epoch, rules, output commitments and amounts, and admission grants. Output bodies may be delivered separately; the recipient checks the corresponding commitment, position, and ownership before saving an output. The certificate authenticates the issuer's guarantee; it is neither an additional asset nor an already registered public obligation.

We distinguish *certified receipt*, when the wallet verifies and locally records an output, *public execution*, when a committed command consumes inputs and creates outputs, and *liability closure*, when a missing source is fulfilled or compensated. Receipt supplies evidence for a subsequent request; that request undergoes its own authorization and resource checks. The implementation's `Settled` flag denotes successful public execution and may coexist with nonzero `Pending`. Source arrival below means successful public execution of the source transaction, rather than receipt of its bytes.

TABLE I
PRINCIPAL NOTATION.

Symbol	Meaning
$G, \mathcal{C}$	Guarantor organization and public committee
T, c_T	Payment and its certified summary fact
(x, i)	Output identifier and spend instance
g, B_g	Grant and cumulative authorization
a_T, p_T, Rec_T	Total CAL outputs, gross compensation, and recovered compensation
$\mathcal{O}_x$	Direct obligation for a missing output
(H_b, P_b)	Stable block hash and original part-set header
ρ_ℓ, ρ_p	Committed logical and installed physical revisions

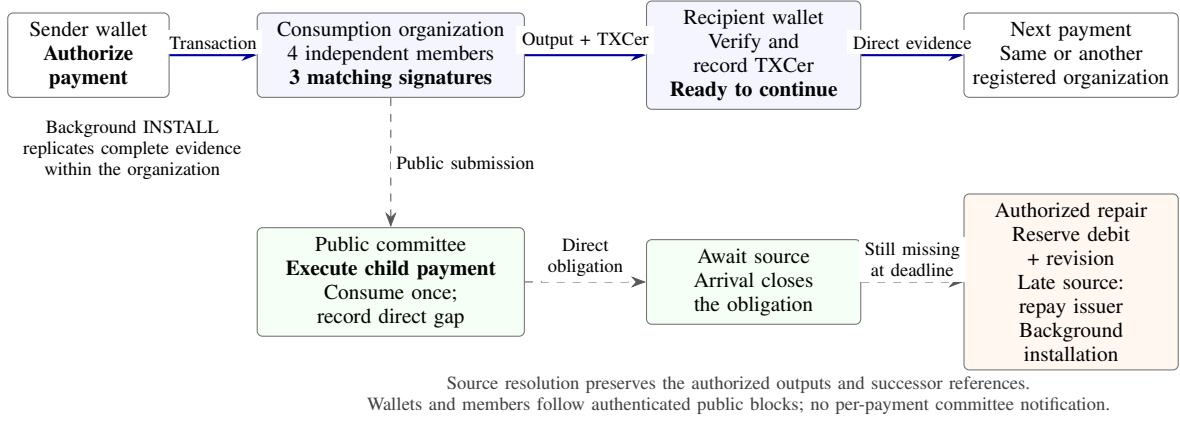


Fig. 1. Continuous payment and public source resolution in Next. Solid arrows form the recipient’s continuation path. Dashed arrows denote background work. Each subsequent payment is independently authorized; an unresolved source is assigned to its direct issuer rather than propagated as an ancestor-waiting requirement. Historical repair is authorized by a later public command; a source that executes after compensation repays the compensating issuer without crediting the recipient again.

C. Three-Layer Design

Certification and continuation. A member atomically reserves the input instances and applicable resources before signing. A CAL grant g authorizes B_g ; each member receives $\lfloor 2B_g/3 \rfloor$ signing capacity, partitioned exclusively among its local execution Workers. A payment reserves a_T , the sum of all its CAL outputs, including change. These overlapping signing capacities constrain commitments rather than creating additional reserves. The collector returns a TXCer after three matching approvals in one parallel request–response round. Recipient verification and atomic local recording complete the fast path; saving complete relay material, member installation (INSTALL), and public submission proceed in the background. Input locks persist independently of budget release.

Source-decoupled public execution. The public command carries the transaction, admission vector, organization authorization, and direct input certificates. The committee reconstructs the certified summary and verifies the authorization without a separate new-output certificate wrapper. It also checks the current consumption state. When a certified input has not yet been publicly created, the committee can atomically record its consumption, register certificate-wide coverage against the issuer, create $\mathcal{O}_x$ and the corresponding gap, and create the child outputs. Registration covers all outputs of the certificate once; a direct obligation arises only for a missing output actually consumed. Execution uses direct evidence rather than recursively collecting ancestor transaction bodies.

Wallets, members, and collectors independently follow authenticated blocks and results. Members restore capacity against their own recorded debits. After observing the successful execution of its own payment, a member restores its CAL debit down to the net compensation it recorded for that payment’s outputs, and applies only the increase over its previous restoration. Because a source’s execution repays every compensated output of that payment in the same transition, this net amount is zero at that point. A reserve loss therefore persists only while its source remains unexecuted. Genesis checks aggregate grants against each backing account. Top-ups

debit accounts outside the protected reserve set and atomically increase the destination balance and grant; authorizations never decrease during an execution.

Compensation, recovery, and historical repair. A missing obligation closes through source arrival or, after a deadline determined by public block time, compensation from its issuer’s reserve. A source that arrives after compensation repays the compensating issuer within its own execution. A repair commits the economic changes together with a validated replacement of the designated Funding/opening and its revision task. Restricted chameleon-hash adaptation preserves the fixed authorization, output references, and complete block identity (H_b, P_b) . RepairBatch groups obligations targeting one historical block and applies their economic effects sequentially within one atomic transaction.

Original command history, committed logical revisions ρ_ℓ , and installed physical revisions ρ_p remain separate. Installation may advance directly to an authorized later version without repeating compensation. Economic replay uses original commands and subsequent repairs; existing successor payments require neither new signatures nor re-execution.

D. A Continuous-Payment Example and Design Goals

Alice pays Bob 100 CAL through G_A , creating output x routed to G_B . Bob verifies its TXCer and obtains a certificate for a payment to Carol. If Bob’s payment executes first, the committee consumes $(x, 0)$, records a 100-CAL obligation against G_A , and creates Bob’s payment outputs. Carol can continue from this payment without obtaining Alice’s transaction body.

If Alice’s payment executes successfully before compensation, it closes the obligation without reviving consumed instance 0. Once public time reaches the deadline, an obligation that remains Open can instead be closed by an authorized repair that debits G_A and replaces the funding reference in Bob’s payment; Carol receives no second principal credit. If Alice’s payment executes after compensation, it consumes its own inputs under the same rules and repays G_A the

100 CAL it advanced. No second output is created for Bob, whose payment was already funded, and instance $(x, 0)$ stays consumed. If Alice’s payment never executes, G_A bears the loss.

The design targets continuation before public execution, funded direct obligations, and preservation of existing payments across source resolution. It connects the recipient’s certified evidence to a bounded public responsibility and an auditable repair path, while keeping historical installation outside the normal payment’s waiting conditions.

IV. PROTOCOL DESIGN

Next certifies fixed payment authorizations and outputs while permitting their sources to remain unresolved. A recipient uses a TXCer as evidence when authorizing a successor. Public execution atomically consumes a missing certified input and records its direct issuer obligation. An authorized repair commits the obligation’s economic closure together with an exact replacement funding representation. Historical installation follows this commitment and never re-executes the payment.

A. Identity, Certification, and Delivery

Let K_T denote the encoded payment body and input claims, C_T its input commitments, and Σ_T its output summary:

$$\begin{aligned} K_T &= \text{Encode}(T.\text{Body}, T.\text{Claims}), \\ \text{id}(T) &= H(\text{TX_V3}, K_T, C_T), \\ c_T &= H(\text{DIRECT_CERT_V3}, \text{Encode}(\Sigma_T)). \end{aligned} \quad (1)$$

Owner signatures authenticate $\text{id}(T)$ without entering its computation. Each output identifier binds this identity, the network, and its output index. Funding references and openings are separately encoded, permitting a restricted replacement without changing the authorized claims or outputs. Initial submission requires `OriginalFunding` referencing each claimed input and a valid opening. Subsequent repair preserves the body, claims, commitments, and owner signatures. Static verification caches therefore key complete command bytes, rather than treating equal transaction identities as interchangeable verification results.

For a new approval, a member checks ownership, routes, direct input certificates, value conservation, and the admission vector. In one local transaction, it rechecks current input locks, final fee inputs, and the designated Worker’s resource slices, then records the approval and exact debits. CAL reservation equals a_T , including change, even when all principal inputs are final: every newly certified output may circulate before this payment executes. The member signs after the transaction commits. Retries reuse the approval; releasing resource capacity never clears its consumption locks.

The collector sends one canonical request to four members and returns after verifying three signatures over the same c_T . Different valid signature subsets retain the same fact identity. The recipient checks the TXCer against the output body, index, owner, and trusted configuration, then records the coin atomically. It can immediately construct a successor

request. Each hop supplies its direct input certificates, rather than embedding the ancestor transaction chain.

Complete relay material is saved while bounded early tasks permit `INSTALL` and public submission to proceed independently. Installation records the certified consumption locally without creating another approval or budget debit. Members schedule backup submission relative to their first installation; repeated installation cannot postpone that deadline. HTTP acceptance leaves the outbox pending. Authenticated public completion removes it and takes precedence over late saves. This propagation path depends on complete submission material remaining available to a continuing sender.

The public `DirectSubmission` contains the transaction, admission vector, organization `SpendQC`, and required direct input certificates. The committee reconstructs Σ_T and verifies the same authorization. Omitting a separate new-output TXCer wrapper changes the envelope, not the authorization requirement.

B. Atomic Execution with Direct Obligations

Public execution separates certificate-wide coverage from an actual missing-input obligation. On the first missing use of a parent fact c , `register` reserves its entire certified CAL amount and creates a Promise for every output. The coverage record maintains

$$\text{Original}_c = \text{Discharged}_c + \text{Paid}_c + \text{Remaining}_c, \quad (2)$$

where Paid_c is a gross cumulative counter. A separate counter $\text{Rec}_c \leq \text{Paid}_c$ records the part repaid by source execution. Grant usage must satisfy $\text{Reserved}_g + \text{Spent}_g \leq B_g$, where Spent_g sums $\text{Paid}_c - \text{Rec}_c$ over the certificates under g . Registration increases public grant usage without debiting the reserve account. An effective compensation debits the account and increases Spent_g ; a repayment credits the same account and decreases Spent_g by the same amount. Neither changes B_g . Only missing outputs actually consumed create direct obligations and increase the public gap. Thus, consuming one missing 40-CAL output of a 40/60 certificate reserves 100 CAL but creates a 40-CAL gap. The unused Promise remains covered. Execution does not register coverage for its own new outputs merely because they have been certified.

Figure 2 specifies the public transition. Its prestate includes earlier successful commands in the same block. Static authorization may be cached; consumption, balances, grants, and obligations are evaluated against this current state. All checks and changes occur in a private overlay. Failure discards the entire transition, including provisional fees and coverage.

Each obligation binds the consumed output, parent certificate fact, issuer, amount, consuming transaction, and historical input location. Its deadline is anchored by public block time. The consuming payment records its Pending input obligations and fee progress. User-funded fees consume authenticated final FUEL, return excess input value as change, and escrow the signed maximum. The maximum covers registration, settlement, closure, and one repair charge per certified input. Stage and obligation identities prevent duplicate charging; unused escrow is refunded to the original payer when its input obligations close.

Require: Verified submission T , current state S , public time

- 1: If T is already settled, return no new effect
- 2: $O \leftarrow \text{Overlay}(S)$
- 3: Check intent, grants, and work; fund fee escrow in O
- 4: **for** each CAL input (x, i) **do**
- 5: Require (x, i) to be unconsumed
- 6: **if** a final creation for (x, i) exists **then**
- 7: Check its body and evidence against the claim
- 8: **else**
- 9: Require $i = 0$ and a matching direct TXCer c
- 10: Register all of c 's coverage if unseen
- 11: Create unique O_x ; add gap and Pending
- 12: **end if**
- 13: Record consumption of (x, i) in O
- 14: **end for**
- 15: Require equal total CAL input and output values
- 16: Record T settled; advance its fee stages
- 17: Resolve each output: fulfill an Open obligation; repay the issuer for a compensated one and mark it Recovered; otherwise fulfill any registered Promise. Create final instance 0 unless recovered
- 18: Return all changes and results as one transition

Fig. 2. Public payment execution. Any failed check rejects all provisional changes. Output resolution follows the branches described in Section IV-C.

This transition makes the child outputs final even while some inputs remain obligations. A further transaction can consume those outputs under the ordinary final-input checks. The unresolved responsibility stays with the directly certified source rather than being propagated as a recursive execution prerequisite.

C. Source Closure, Recovery, and Cumulative Reconciliation

A source transaction arrives through normal payment execution, including validation and consumption of its own inputs. Each of its outputs is then resolved according to the state of its obligation, within the source's own transition. If the obligation is Open, arrival fulfills its Promise, discharges the associated coverage, and decreases the consumer's Pending count. If the output was never consumed, any registered Promise is fulfilled and the output becomes final instance 0. If the obligation was already compensated, the consumer was funded by a reserve debit, so the arrival repays that debit instead of creating a second output: the same amount is credited to the issuer account that paid it, the obligation becomes Recovered, and the repayment counter and the grant's Spent are updated together. In every branch the previously consumed instance 0 stays consumed. A recovered output has no final creation and cannot be claimed again. Recovery adds no authorization, since it returns capacity that the compensation had spent. The source's own inputs may introduce new direct obligations; these are admitted against their own direct issuers and are checked within the same atomic payment transition. No ancestry is traced, and no repayment can be revoked.

Members reconstruct progress from authenticated consecutive blocks and results. Only an existing local approval supplies a debit to restore. For its CAL debit, member m

computes

$$t_{m,T} = \begin{cases} a_T - (\hat{p}_{m,T} - \widehat{\text{Rec}}_{m,T}), & \text{if settled,} \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

$$\delta_{m,T} = t_{m,T} - \text{Applied}_{m,T},$$

where “settled” means that the member has observed its own payment execute successfully; $\hat{p}_{m,T}$ is the compensation recorded locally for that approval's outputs and $\widehat{\text{Rec}}_{m,T}$ the repayment recorded locally. On observing the execution of its own payment, the member checks that the repayment reported in the authenticated result covers $\hat{p}_{m,T}$ and sets $\widehat{\text{Rec}}_{m,T} = \hat{p}_{m,T}$: that execution repays every compensated output of T , so the member's recorded loss is repaid in full. A member that approved T only after following the compensation recorded none ($\hat{p}_{m,T} = 0$) and recovers nothing beyond its own debit. After checking bounds and monotonicity, the member transfers $\delta_{m,T}$ from Reserved to Available in the original Worker and updates Applied. The payment's own Pending inputs do not delay CAL release: their issuers carry those liabilities. Work resources wait for fee closure; actual FUEL expenditure remains deducted. Repeated blocks and late material cannot repeat credits. An unexecuted source receives no automatic release merely because some outputs were compensated. A wallet records a recovered output as a terminal observation rather than a coin: it carries no certificate, no final creation, and no balance, and a later copy of the old TXCer does not overwrite it.

An authorized reserve increase debits an external, unprotected CAL account, credits the reserve, and increases its grant atomically. Each member adds the difference between the new and old $\lfloor 2B_g/3 \rfloor$ shares without resetting Reserved or approvals. Aggregate backing checks cover all grants sharing an account.

D. Constructing and Executing Repair Batches

A RepairBatch targets one historical block and contains 1–32 canonically ordered items. Each item gives an output identifier, transaction index, input index, and new input opening. Obligations and slots must be unique. The batch fixes the base revision and the previous and replacement canonical-body digests; success advances the logical revision from Base to Base + 1. Amounts, issuers, and replacement references are derived from trusted obligations rather than supplied as editable payment data.

A constructor reads the latest committed logical representation, independently of physical installation. Before requesting adaptation shares, it sorts eligible obligations and sequentially simulates the existing compensation rule in a temporary overlay. Successful applied items contribute their changes to that simulation. Qualification reads the original committed view, not simulated closure records. The final selected set is checked again as a fixed batch. These checks avoid requesting proofs for a batch already known to exceed its account or fee resources; they reserve no funds.

For every item, the constructor derives a reserve-debit identity from the network and consumed output, then replaces its OriginalFunding slot with the corresponding reserve

Require: Batch B , state S , execution height and time

- 1: $b \leftarrow \text{BatchID}(B)$
- 2: If successful Task b exists, return no new effect
- 3: Load Canonical; check older height, base, previous digest
- 4: Check item order and unique obligations and slots
- 5: Require each obligation Open, due, and location-matched
- 6: Reconstruct the permitted combined funding replacements
- 7: Require the canonical candidate-body digest to equal B .Next
- 8: Verify non-target bytes, lengths, and complete block identity
- 9: $O \leftarrow \text{Overlay}(S)$; initialize result
- 10: **for** each item in canonical order **do**
- 11: Evaluate compensation against O ; require Applied
- 12: Merge changes into O ; append effect and fee outputs
- 13: **end for**
- 14: Record shared per-obligation completion identities
- 15: Add one logical revision, Task, and queue entry to O
- 16: Return the transition and execution result for commitment

Fig. 3. Atomic repair-batch execution. Failure rejects the whole batch; only a successful transition is committed. Batch identity covers the complete canonical command, including its proof data.

reference and opening. Multiple slots in one transaction are patched into the same copy. Serialization lengths and all non-target bytes remain unchanged, including prior authorized replacements. Input adaptation is performed per slot. The final combined body determines the affected block parts, each adapted once; unchanged parts retain their commitments and proofs.

Share endpoints are bound to fixed committee identities. Participants independently reconstruct and economically check the requested fixed batch. The collector tests bounded three-member candidates with actual cryptographic combination, accepting a candidate only when every affected part verifies. Malformed or invalid responses eliminate candidates, not three valid contributions from other members. Successful combination cancels outstanding requests. Openings establish commitment compatibility; public execution supplies the repair authorization.

Public execution repeats all checks against its current state (Fig. 3). Sequential evaluation within one overlay handles shared accounts and fee escrows: the last closed input may complete its consumer's fees and produce a refund. A changed base, closed item, or insufficient resource rejects the entire batch. Exact successful replays are recognized before base validation; different batches and the compatible single-repair entry share obligation terminal states, preventing another debit through repackaging. At a fixed prestate, time, and item order, batching groups the same per-obligation economic steps into one transition; command and revision identities remain specific to the chosen packaging.

The authenticated result contains per-item effects identifying parent and consumer facts, amount, and input position, plus top-level fee outputs. Members apply the relevant per-item effects and their cursor atomically, locating original approvals for Paid and Pending updates. Wallets apply each fee output once. Compensation never adds another principal credit to an already executed successor. Recovery is not a batch operation: when the sources of batched obligations arrive, each repayment is reported in that source's own execution result, and the batch identity plays no role.

E. Identity-Preserving Installation and Replay

Input commitments preserve transaction identity; part commitments preserve the complete block identity (H_b, P_b) , where P_b contains the part count and Merkle root. Changed parts receive valid openings to their original commitments. Original consensus parts use opening 1 and revision 0. Candidate, lock, and commit checks compare the complete block identity rather than the stable header hash alone.

The public revision and Task authorize an exact new body and its part evidence. An installer copies this material from one committed read view before acquiring the physical revision lock. For target revision r , a lower physical version is verified and installed; an equal version requires identical material; a higher version covers the old task without rollback. The original block is retained, and parts and local version metadata are written atomically. Installation may skip intermediate physical versions while retaining every economic command.

Queue progress remains global and consecutive. For entries A_1, B_1, A_2 , installing A_2 while handling A_1 does not complete B_1 ; the cursor advances only across processed entries. Logical execution reads committed Canonical revisions, so different installation schedules leave the same economic history. Replay uses original blocks followed by their repair commands, reconstructing application hashes for that same public command sequence. Existing successors retain their authorizations, output references, and block identities.

An append-only variant could close the economic account without revising history. The implemented protocol couples compensation with a validated revision, so progress requires available adaptation contributions; the accounting invariants do not rely on the adapter's proposed output. The historical interface offers a further capability: after repair, the stored block carries a funding representation that names the reserve debit advanced for the consumer and still verifies against the original commitments. Standard wallets, block synchronization, and replay read original commands; the committed revision is read by repair construction, installation, and local inspection. The revised body records which reserve funded the input at compensation time, not who bears the loss now: a later repayment is a separate economic fact reported in the source's result. Batching shares representation work within a block, and direct installation coalesces historical writes; original commands and logical revisions remain retained. Neither step adds a round to normal certification or requires a recipient to await historical installation.

The supplementary methods compare this representation with an append-only compensation design at the level of capabilities and dependencies.

V. SECURITY ANALYSIS

We analyze finite executions of the protocol in Section IV. Each fixed four-member organization and the independent, equally weighted four-validator committee have at most one statically Byzantine participant; certificates require three distinct signatures. Honest participants share legal genesis and a registry assigning each route one active configuration and lock domain. Signing state is atomic and never forgotten or rolled

back; application transitions are deterministic and atomic, and followers apply authenticated consecutive blocks with state and cursor updated atomically. Claims concern states at these commit boundaries. We condition on unforgeable signatures and binding canonical digests and original-part commitments. Adapter output is treated as an untrusted candidate: a revision takes effect only through a committed repair whose guards derive the permitted slot, amount, and issuer and verify every non-target byte and the complete block identity; installation accepts only bytes equal to the committed Task body. Threshold adaptation supplies commitment-compatible candidates, and its availability governs repair progress. Delays are arbitrary. We first establish the original-execution interface, then connect certification, reserve coverage, and repair. The consensus argument specializes the Tendermint locking interface [18] to complete block identities.

A. Original Execution and Certified Consumption

A *polka* is a same-round quorum of prevotes for a complete block identity or nil.

Lemma 1 (Original-execution agreement): Under the locking and validity rules of the consensus interface, one original block body is admitted per committed height, including its application context.

Proof: With original opening 1, a part leaf hashes its context-bound message representative. Binding and the Merkle proof fix its bytes under the original part-set header. Candidate bodies are reconstructed from their parts or locally paired with parts; Locked and Valid transfers preserve this pairing. Proposal, proof-of-lock (POL), lock, and commit comparisons use (H_b, P_b) , and changing a download target clears the old body. A matching current-round polka may trigger precommit before Proposal arrives; the existing round and step guards still prevent a second precommit.

Same-round conflicting commits would require an honest validator to precommit twice. For a commit of value v in round r , take its two honest signers and suppose a conflicting polka exists later in round number. Choose the smallest such round $s > r$ and its earliest certificate formation. Its quorum intersects those signers. That signer's vote requires an earlier unlocking justification: a lower conflicting round contradicts minimality, and an earlier same-round polka contradicts the chosen formation event. Include nil polkas that authorize unlocking. The argument uses signature events, not observation of the completed commit certificate. Honest rounds advance monotonically, so a higher-round vote by a commit signer follows its own earlier lock, even if the commit certificate is assembled later. The validity of a vote cast for a locked value traces to earlier lock rounds and ultimately to actual proposal validation. Finalize follows a commit matching the candidate's complete block identity. Original synchronization and replay preserve the body, height, and time, so retries supply the same execution material. ■

Lemma 2 (Authenticated, unique consumption): A TXCer fixes its outputs and issuer. No two valid certificates authorize conflicting consumption of the same (x, i) , and public execution consumes that instance at most once.

Proof: Binding fixes the certified summary and owner-authorized transaction. The fixed route sends competing requests to one organization. Two three-member quorums intersect in at least two members, including an honest member whose atomic input lock precedes signing. That member cannot approve both conflicts. Budget release preserves the lock. Final-input and certificate-input execution consult the same spend key; fact replay produces neither another consumption nor another fee charge. A recovered output has no final creation and its instance 0 stays consumed, so no later claim on (x, i) can succeed. ■

B. Covering Public and Hidden Commitments

For a complete CAL grant g , cumulative authorization B_g gives each member $s(B_g) = \lfloor 2B_g/3 \rfloor$ capacity. Let C_g contain every distinct fact whose three signatures have existed by the current prefix. It includes unassembled and hidden certificates and never deletes settled facts. The unique CAL allocation binds each fact to one configuration, ResourceKey, and Grant ID. A fact c has output cap a_c , the sum of its CAL outputs. For $c \in C_g$, let p_c be the cumulative compensation paid for those outputs, Rec_c the part repaid by source execution, and z_c indicate that the payment of c has executed successfully. The net compensation is $\bar{p}_c = p_c - \text{Rec}_c$. Define

$$w_c = \begin{cases} a_c, & z_c = 0, \\ \bar{p}_c, & z_c = 1, \end{cases} \quad Q_g = \sum_{c \in C_g} w_c. \quad (4)$$

Lemma 3 (Source repayment): A compensated output leaves the Repaired state only inside the successful execution of its source, in the same transition that sets $z_c = 1$, and the repayment is credited to the account that was debited. Hence $0 \leq \text{Rec}_c \leq p_c \leq a_c$, $\text{Rec}_c = 0$ while $z_c = 0$, and $\text{Rec}_c = p_c$ once $z_c = 1$. In particular $\bar{p}_c = p_c$ while the source is pending and $\bar{p}_c = 0$ afterwards, so no net compensation survives the execution of the source.

Proof: Each registered promise leaves Open at most once, and compensation applies only to a funded Open obligation, so $p_c \leq a_c$. The only transition leaving Repaired is the output-resolution step of a payment whose fact equals the obligation's certificate fact and whose issuer equals the obligation's issuer; it executes in the same overlay that records the payment as settled, so z_c and the repayment change together. That step resolves every output of the source that has an obligation: Open becomes Fulfilled and Repaired becomes Recovered. Outputs without an obligation incur no compensation. Afterwards every promise of c is closed, so no new compensation can arise, and a repeated submission is a no-op. Compensation debits and repayment credits both use the issuer account stored in the obligation, which is the account of the CAL allocation of c . ■

For any local approval, define $r_{m,c} = a_c - \text{Applied}_{m,c}$, where Applied is its cumulative CAL restoration.

Lemma 4 (Residual capacity): For every fact $c \in C_g$ and every honest member m that approved c , at every prefix, $r_{m,c} \geq w_c$.

Proof: Until m observes the successful execution of c 's payment, its CAL restoration for c is zero, so $r_{m,c} = a_c \geq w_c$

whatever the compensation state. After it observes that execution, m checks that the reported repayment covers its recorded compensation and sets its recorded repayment equal to it. The restoration target is a_c minus a recorded net compensation of zero, so $r_{m,c} = 0$, while $w_c = \bar{p}_c = 0$ by Lemma 3. A member that has not yet observed its payment's public execution retains $r_{m,c} = a_c \geq 0 = w_c$. The statement covers a member that approved after following a compensation: it recorded none and restores only its own debit. ■

Theorem 1 (Potential liabilities are funded): For every CAL grant, $Q_g \leq B_g$. For each reserve account x , its total open gap D_x does not exceed its balance A_x .

Proof: Let $\mathcal{A}_{m,g}$ include all successful approvals under g , even without a QC, and $L_{m,g} = \sum_{c \in \mathcal{A}_{m,g}} r_{m,c}$ across Workers. Approval and restoration transfer capacity between Available and Reserved. Top-ups add the difference in member shares, so $L_{m,g} \leq s(B_{m,g}) \leq s(B_g)$ for the locally observed authorization $B_{m,g}$; grants only increase. Choose any fixed three honest members H_g of this organization. A QC has three distinct signers and at most one member lies outside H_g , so the set Sig_c of members of H_g that signed c has at least two elements. Honest members sign only after approving, so Lemma 4 gives $r_{m,c} \geq w_c$ for $m \in \text{Sig}_c$, and

$$\begin{aligned} 2Q_g &\leq \sum_{m \in H_g} \sum_{\substack{c \in C_g \\ m \in \text{Sig}_c}} r_{m,c} \\ &\leq \sum_{m \in H_g} L_{m,g} \leq 3[2B_g/3] \leq 2B_g. \end{aligned} \quad (5)$$

Let R_c and d_c be registered remaining and discharged coverage, with $p_c = d_c = R_c = 0$ before registration. Registered records satisfy $a_c = p_c + d_c + R_c$, and $z_c = 1$ implies $R_c = 0$. Therefore $R_c \leq w_c - \bar{p}_c$: while $z_c = 0$ we have $\text{Rec}_c = 0$, so $R_c \leq a_c - p_c = w_c - \bar{p}_c$, and once $z_c = 1$ both sides are zero. Define $S_g = \sum \bar{p}_c$, $V_g = \sum R_c$, and $E_g = Q_g - S_g$, with sums over C_g . Registration, compensation, repayment, and discharge maintain public $\text{Usage.Spent} = S_g$ and $\text{Usage.Reserved} = V_g$.

Let $\mathcal{G}_x$ contain all grants backed by x . Their certified CAL account equals the debited issuer account. Genesis establishes $\sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x$. External, non-protected top-ups increase both sides equally. Compensation increases S_g and decreases A_x by the same amount; by Lemma 3, repayment decreases S_g and increases A_x by the same amount. No other protocol operation withdraws CAL reserves. Each Open obligation corresponds to a distinct remaining promise, giving

$$\begin{aligned} D_x &\leq \sum_{g \in \mathcal{G}_x} V_g \leq \sum_{g \in \mathcal{G}_x} E_g \\ &\leq \sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x. \end{aligned} \quad (6)$$

Gross Paid and its recovery are each counted once; E_g is a risk bound, not available member capacity. ■

CAL admission consequence. At the execution prestate, including earlier successful block commands, let N_g be the distinct, valid, unregistered parent facts under g required by matching, unconsumed missing inputs. They have $w_c = a_c$ and

$p_c = R_c = 0$. Every registered fact satisfies $R_c + \bar{p}_c \leq w_c$, and the facts are disjoint, so

$$V_g + S_g + \sum_{c \in N_g} a_c \leq Q_g \leq B_g. \quad (7)$$

Repeated uses of one certificate contribute one term, not one term per input. Sequential registration sees earlier reservations, and every partial sum remains within this bound. First registration therefore cannot fail solely for CAL authorization capacity, even when it precedes a repayment in the same transition, since the repayment only lowers S_g . Likewise, an Open amount $a \leq D_x \leq A_x$ has sufficient reserve principal. Other resource and fee checks remain separate.

C. Asset Conservation and Delay Neutrality

Theorem 2 (CAL conservation): Let U be the total value of final, unspent CAL instances, A the total CAL account balance, and D the total value of Open gaps. Every execution prefix satisfies

$$U + A - D = C_0, \quad (8)$$

where C_0 includes genesis outputs and all accounts, including top-up sources.

Proof: A payment consumes final value F and missing value M , producing $F + M$. Let J be its arriving outputs whose obligations were Open, and R those whose obligations were compensated. Outputs in J were already consumed, so they close gaps rather than add unspent value, and outputs in R receive no creation record. Hence $\Delta U = M - J - R$, $\Delta D = M - J$, and the repayment gives $\Delta A = R$, so $\Delta(U + A - D) = 0$. Compensation of a gives $\Delta A = \Delta D = -a$ and $\Delta U = 0$. Top-ups transfer between included accounts. Other transitions preserve this projection. ■

FUEL uses separate accounting. Each escrow maximum equals Held plus cumulative Rewards, Burned, and Refunded. The global conserved sum counts unspent FUEL, ordinary accounts, current Held, actual reward balances, and cumulative burn. Unique stage and obligation identities transfer value once; refunded outputs and credited rewards are not counted again through cumulative escrow fields. This includes user-funded and authorized organization-funded fees.

Conservation fixes totals but not who holds them. The next result shows that delay does not change the final holders.

Theorem 3 (Delay neutrality): Let $\mathcal{T}$ be a finite set of payments such that (i) no two consume the same instance or fee input, (ii) the relation “consumes an output created by” is acyclic, and (iii) every certificate input of a payment in $\mathcal{T}$ certifies an output of a payment in $\mathcal{T}$. Let $\mathcal{U}$ be a finite set of authorized reserve increases. Start from a clean genesis state with no coverage, promise, or obligation records and zero CAL Reserved and Spent. Let ρ_0 execute $\mathcal{T}$ in an order consistent with (ii), applying $\mathcal{U}$, and let ρ be any execution consisting of the payments of $\mathcal{T}$, the increases of $\mathcal{U}$, and compensations of due Open obligations, in any order. Suppose every payment and increase succeeds exactly once in both executions. Then at the end of ρ and of ρ_0 :

- 1) the final, unspent CAL instances coincide, with the same owners and amounts;

- 2) every reserve account holds the same CAL balance;
- 3) every CAL grant has the same authorization and Reserved=Spent= 0, every obligation is Fulfilled or Recovered, and Paid_c = Rec_c for every certificate.

Fees and FUEL are not compared.

Proof: Holders. By (i), both executions consume the same instances. In ρ , an output consumed before its source executes receives an obligation; when the source executes, the output is either Fulfilled, with a creation record that is already consumed, or Recovered, with no creation record. Either way it is consumed. Outputs consumed by no payment of $\mathcal{T}$ are created as final and unspent in both executions. Hence the unspent instances are the genesis and $\mathcal{T}$ outputs not consumed by $\mathcal{T}$, in both.

Reserves. For a reserve account x let $\hat{A}_x$ be its balance plus the amounts of Repaired obligations issued by x . Compensation lowers A_x and raises the Repaired total by the same amount; by Lemma 3, repayment does the reverse on the same account; creating or fulfilling an obligation changes neither term. As in Theorem 1, only top-ups, compensation, and repayment change reserve balances. Therefore $\hat{A}_x = A_x(0) + \tau_x$ at every prefix, where τ_x is the total of increases credited to x so far. An obligation exists only for an output consumed before its source executes. By (iii) that source executes in ρ , and its execution leaves the obligation Fulfilled or Recovered. So at the end no obligation is Open or Repaired, $A_x = A_x(0) + \tau_x$, and this is also the value in ρ_0 , where no compensation occurs.

Usage. Every registered certificate is used by a missing input, so by (iii) its source executes in ρ . That execution closes all of its promises, so Remaining= 0, and Lemma 3 gives Paid_c = Rec_c. Hence Reserved= $\sum$ Remaining= 0 and Spent= $\sum$ (Paid_c - Rec_c) = 0. Authorizations change only through $\mathcal{U}$. ■

Intermediate states differ: while a compensated source is pending, its issuer's balance is lower by the compensated amount, which Theorem 1 covers. The theorem is not a claim of incentive compatibility, and it says nothing about a source that never executes, whose compensated amount remains a net loss of its issuer.

D. Authorized Repair and Representation Independence

Lemma 5 (Constrained repair authority): Every newly effective repair item closes its unique, due, execution-prestate Open obligation. A successful repair changes no transaction bytes outside its authorized funding slots, even when adaptation outputs are adversarial.

Proof: Let N be the RSA modulus, e its public exponent, and r an opening in $\mathbb{Z}_N^*$. The domain-separated representative FDH(ctx, m) binds the message and its repair context. The relation

$$C = \text{FDH}(ctx, m)r^e = \text{FDH}(ctx, m')r'^e \pmod{N} \quad (9)$$

establishes compatibility, not fresh approval; a published opening ratio can be reused for the same message relation. Public guards independently derive the slot, issuer, amount, and reserve-debit identity, check current base and body digests, and verify all non-target bytes and complete block commitments.

Invalid candidates leave no changes. Exact successful replays are no-ops; repackaged commands encounter the shared obligation terminal state. Economic effects and the authorized revision Task commit together. The adapter neither writes ledger state nor forges committee authentication. The argument uses its verified output, not evidence of fresh threshold participation. ■

Let π_{econ} retain public assets, consumption, coverage, obligations, grant usage, and fee states, omitting revision and installation metadata.

Lemma 6 (Batch equivalence and installation independence): At its commit boundary, an accepted batch has the economic effect of its ordered individual compensation steps starting from the same state, at the same time, without intervening commands. For the same public history, legal physical installation schedules have identical economic projections.

Proof: Unique obligations and slots are canonically ordered. Each step reads the shared overlay, including earlier account, Pending, and fee changes. Failure discards all steps; success commits their aggregate effects with one revision. Base is a revision number; Previous and Next authenticate body digests. The new revision is Base+1.

Canonical reads committed logical revision ρ_ℓ . An exact Task authorizes body and parts at revision $r \leq \rho_\ell$. Installation advances if $\rho_p < r$, checks exact equality if $\rho_p = r$, and treats $\rho_p > r$ as already covered without rollback. It changes no economic variable. Queue progress crosses only completed entries: installing A_2 at A_1 cannot skip intervening B_1 . Therefore, for legal schedules μ_1, μ_2 over one public prefix,

$$\pi_{\text{econ}}(S_{\mu_1}) = \pi_{\text{econ}}(S_{\mu_2}). \quad (10)$$

Batch and single-command histories can have different application hashes; deterministic replay of the same original command history reproduces its application hash at every height. Repairs preserve fixed transaction fields, authorizations, output identities, successor references, and complete block identities.

For a follower starting from the same local state, configuration, and cursor, equal fixed payment fields and authenticated results induce equal economic updates whenever both representations decode. Mutable Funding/opening does not select Paid, Pending, or fee effects. Repair commands are authenticated in full; execution-result commitments authenticate their per-item effects and fee outputs. This separates economic observation from independent verification of the latest mutable historical body. ■

E. Protected Continuation and Progress

Define FreshRecv(w, o, c) as authenticated receipt by an honest owner who has not previously authorized consumption of that instance. Freshness is a history predicate, not inferred from successful receipt alone.

Theorem 4 (Protected continuation): Following FreshRecv, the certified output cannot be reinterpreted or consumed without owner authorization. A certified, publicly executed successor consumes a final source or atomically establishes a funded direct obligation. Source closure and repair preserve

its established principal effects without successor re-signing or re-execution.

Proof: Lemma 2 fixes ownership and excludes conflicting consumption. Atomic execution couples each missing consumption with its obligation and outputs. Theorem 1 supplies funding; Theorem 2 accounts for closure and late sources. Lemma 1 identifies the executed history, and Lemmas 5–6 restrict later changes. Induction over the joint local/public transitions composes these properties; INSTALL retains local consumption effects but does not change public assets. Repair may change Pending, fees, and reserve balances without repeating successor principal. ■

Continuation requires responsive honest quorums, available evidence, sufficient fees and Worker/work capacity, and fair delivery and inclusion. Repair additionally requires advancing public time and available adaptation contributions. Partial approvals and unused promises can retain capacity; timeouts alone never authorize unlocking. These are progress conditions, not a universal completion deadline.

Data availability and guarantor risk. Three questions are separate. *Safety:* Theorem 1 funds every certificate ever signed, whether or not its source is submitted. *Progress:* a source can progress when an honest relay holds its complete submission material (transaction, admission vector, organization authorization, direct input certificates) and the remaining fee, capacity, quorum, and fair-scheduling conditions hold; delivery time is unbounded. A deadline can therefore pass for ordinary reasons such as congestion or a short outage. Compensation then commits when adaptation contributions are available: the issuer advances the missing principal and is repaid when the source executes. When all sources of the payment set execute, Theorem 3 shows final holders and reserves unchanged. *Economics:* the member approval endpoint authenticates owner, budget, and inputs but not the caller’s role, so any client can act as collector. If that client never installs or submits the assembled material and no other relay obtains it, the source stays unexecuted and, once compensated, its issuer bears the amount as an unrecovered loss. Reserve sizing reflects the expected rate of such sources; fees are not priced as insurance against them.

VI. IMPLEMENTATION AND EVALUATION

Next’s evaluation connects recipient-side continuation to funded closure and identity-preserving repair. We examine dependent payments first, then source failures, repair costs, organization boundaries, and operating capacity. These measurements exercise the mechanisms of Section IV under the model in Section III; Section V establishes their safety properties under the stated model.

Implementation and measurement. The Go implementation uses atomic state overlays, independent block followers, and CometBFT v0.38.26 modified to enforce complete block identities and authorized historical installation. Threshold RSA adaptation uses CIRCL v1.6.5 with trusted offline dealer initialization. Network experiments run on one Mac Studio with an M4 Max, 16 CPU cores, 64 GiB RAM, macOS 26.6.2, and Go 1.27.1. Four committee processes and, per organization, four members and one gateway yield nine or fourteen

service processes. Latency/capacity trials primarily use in-memory member and committee backends and bbolt wallets with NoSync; signatures and atomic state updates remain enabled. Source-repair and batch-integration trials use disk-backed committee application state and BlockStore. Each run starts from fresh genesis. Current continuation and recovery trials use final user FUEL. Storage, paired-background, and short-throughput controls use authorized organization-paid FUEL. The archived continuation, operating-range, and batch-integration series use authorized fee sponsorship. The supplement records the build and settings for each series; batch integration is reported separately from the earlier structural measurements.

Certified-receipt latency generally starts at the first actual wallet HTTP send and ends after recipient verification and atomic local recording. Capital/source-failure drivers include a small amount of pre-send construction and are compared within their own series. Public and member completion are observation endpoints, not internal commit timestamps. Completion throughput includes final background drain. Cross-site ACK timing uses the sender’s monotonic clock and includes the return path.

A. Continuous Spending Without Per-Hop Settlement

A 100-hop payment chain reaches its last certified receipt in a median 158.846 ms, compared with 65.204 s when successive hops wait for public confirmation. Three wallets transfer 100 CAL in rotation, constructing each payment online from its predecessor’s output and paying fees from their own final FUEL. Three repetitions per mode produce six runs and 600 payments on the nine-service in-memory configuration. Both modes end at the last certified receipt; the waiting baseline inserts public confirmation waits *between* hops. Committee commit, flush, and gossip settings are 250, 10, and 10 ms, respectively. Under these settings, public execution and its observation cost about 0.65 s per hop in the waiting baseline; certified continuation removes this inter-hop wait.

Figure 4 reports the six whole-chain durations. The fast runs take 158.492–159.905 ms. All 297 fast-mode successor requests consume the preceding certified output and are sent after its certified receipt but before its public completion is observed. A one-input, one-output TXCer is 779 bytes; requests are 1,979 bytes at the first hop and 2,766 bytes thereafter. For this fixed transaction shape, direct evidence size stays constant across all 100 hops. These observations show online dependent spending without inserting a public-confirmation wait on the recipient’s critical path.

Synchronous member commits. Three paired 100-hop trials use a nine-service disk-backed configuration, NoSync wallets, user-paid FUEL, and the same 250/10/10-ms consensus settings. Each pair differs only in member storage (NoSync or synchronous; order N/S, S/N, N/S), followed by a synchronous waiting chain. Median durations are 681.127 ms for NoSync continuation, 4.471 s for synchronous continuation, and 65.013 s for synchronous waiting. Thus certified continuation remains 14.5 times faster than waiting under synchronous member commits. All 900 payments complete with agreeing committee states; the supplement reports each run.

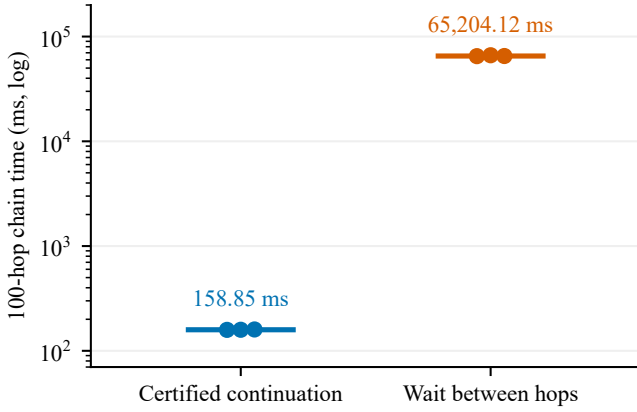


Fig. 4. Time through the last certified receipt of a 100-hop chain. Dots are three independent runs per mode; horizontal bars mark their medians. The logarithmic time axis includes the same endpoint in both modes. Only the waiting baseline requires public confirmation between successive payments.

On the archived delivery-gate build, removing the post-certification delivery gate also shortens continuation under matched offered load. The gated variant waits for three members to install complete material or for authenticated public success, whichever occurs first; it collects no second signature quorum. At 200 payments/s for 30 s, three paired runs per mode each send and complete 6,000 payments. The median of run-level receipt P50s is 1.300 ms with immediate delivery and 2.167 ms with the gate, a 40.0% reduction. Separate 100-hop trials reduce median chain time from 287.983 to 200.356 ms, or 30.43% (Fig. 5). The gain removes a replication/completion wait while preserving the same three-signature authorization.

B. Funded Closure and Source Recovery

Source-recovery matrix. Nine runs use seeds 23, 37, and 59 at withheld-parent rates of 0%, 1%, and 5%. Each run offers 20 two-hop units/s for 60 s (1,200 units, 2,400 payments) against a fixed 60,000-CAL reserve with user-paid FUEL and no top-up. The nine-service deployment uses disk-backed committee application state and BlockStore, in-memory members and gateway, and NoSync wallets. Withheld parents are released after their compensation is committed and installed on all four replicas, so each withheld source is compensated after the 30-s public-time deadline and executes later.

All 21,600 payments close. The 216 compensations (12 per 1% run, 60 per 5% run) are each followed by recovery. At the end of every run no obligation is Open or Repaired, the gap is zero, net Spent and Reserved are zero, and the reserve is back at 60,000 CAL (Table II). Figure 6 shows the three 5% trajectories. The lowest sampled reserve is 59,800–59,900 CAL at 1% and 59,600 CAL at 5%; it reflects the driver’s release schedule, not a reserve-sizing result. Run-level receipt P50s are 1.185–1.251 ms for normal child payments and 1.171–1.207 and 1.176–1.190 ms for withheld parents and their children. After the deadline, the committee commit is observed after a median of 1.003–1.254 s and installation on all four replicas after 1.257–1.761 s (P95 up to 2.015 s), including 250-ms observation polling. Each normal payment spends 94

TABLE II
CLOSING AUDIT OF THE SOURCE-RECOVERY MATRIX. EACH COLUMN IS REPRODUCED BY ALL THREE SEEDS; EACH RUN HAS 2,400 PAYMENTS. RESERVE VALUES ARE CAL, FEE VALUES ARE IN THOUSANDS OF FUEL, AND FEE CAPS SUM THE PER-PAYMENT MAXIMA.

Withheld parents	0%	1%	5%
Repairs / recoveries	0	12	60
Paid / recovered CAL	0	1,200	6,000
Final reserve	60,000	60,000	60,000
Unclosed / net Spent	0	0	0
Total fee caps	2,400	2,400	2,400
Rewards	201.600	201.660	201.900
Burned	24.000	24.000	24.000
Refunded	2,174.400	2,174.340	2,174.100
Remaining Held	0	0	0

FUEL and each repair adds 5; rewards, burn, and refunds sum to the fee caps with zero Held.

Ledger-level regression covers the arrival orders directly: all 24 arrival orders of a four-hop chain, with and without due compensation, under three issuer sequences (one organization, alternating two, and three with a repeated first issuer) give 144 trajectories. A further 48 split-merge trajectories vary all arrival orders of a 40/60 split, two branches, and their merge under two compensation modes. Every step in the 192 trajectories conserves the independently computed asset projection, and every trajectory ends with each issuer’s reserve restored, Spent and Reserved zero, Paid = Rec, and the consumed inputs still consumed.

Two additional fourteen-service controls distinguish permanent loss from delayed recovery. When the parent never arrives, the grandchild retains its 100 CAL while the responsible issuer’s reserve remains 100 CAL lower. When a cross-organization chain arrives grandchild first, the intermediate source closes its own direct obligation; the root issuer’s compensated 100 CAL is recovered when the root later executes. The four committee states agree in both cases. An absent parent’s own fee and work reservations can remain outstanding.

Paired end-state check and interference. Three pairs of fourteen-service runs, ordered normal/repair, repair/normal, and normal/repair, use the initial source-recovery build, before the batch-result tag separation, with committee and member databases on disk and member and wallet NoSync, each send 4,000 payments at 100 payments/s for 40 s. Alongside them, four additional two-hop demonstrations have sources that are delivered normally (normal run) or withheld, compensated after 30 s, and later recovered (repair run). All 4,000 payments complete in all six runs. The final CAL held by each owner role (30,000 and 830,000) and both reserve balances (10^{12}) coincide across the six runs, and the repair runs record 400 CAL paid and 400 CAL recovered. Receipt P50s for normal and repair runs are 1.070/1.127, 1.089/1.108, and 1.083/1.097 ms, and P95s are 26.84/59.23, 29.49/58.44, and 27.41/55.15 ms; completion rates are 97.2–97.8 payments/s in all runs. The final allocation is consistent with Theorem 3. Receipt P50 is 1.3–5.3% higher and P95 is 1.98–2.21 times the paired normal values. Concurrent repair therefore affects the receipt tail in

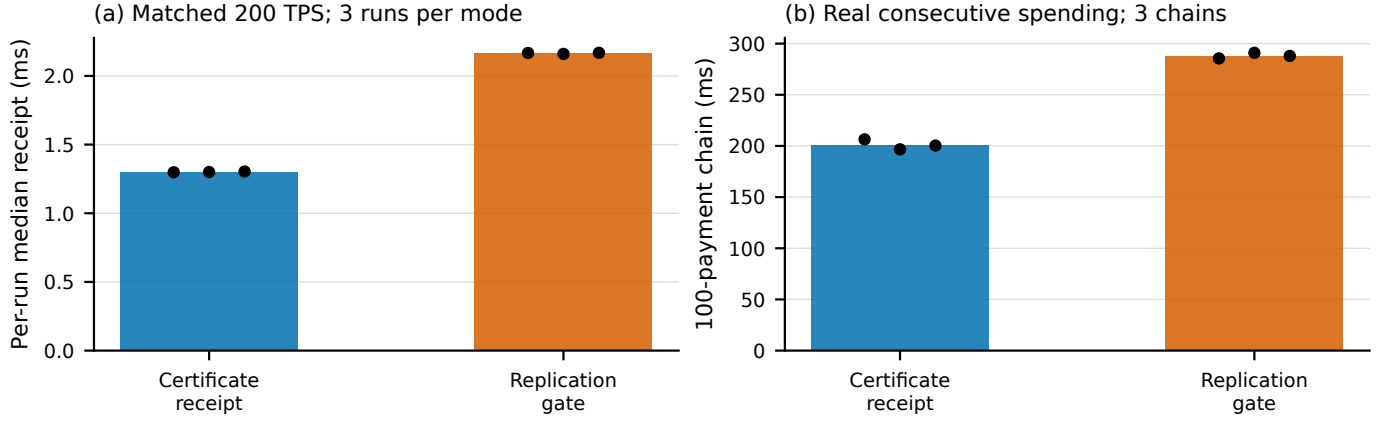


Fig. 5. Delivery-gate effect (ms). Left: run-level receipt P50 at 200 payments/s, 6,000 payments/run, three paired repetitions. Right: whole-chain time for three 100-hop chains per mode. Columns show the median of the three run values; dots show individual runs.

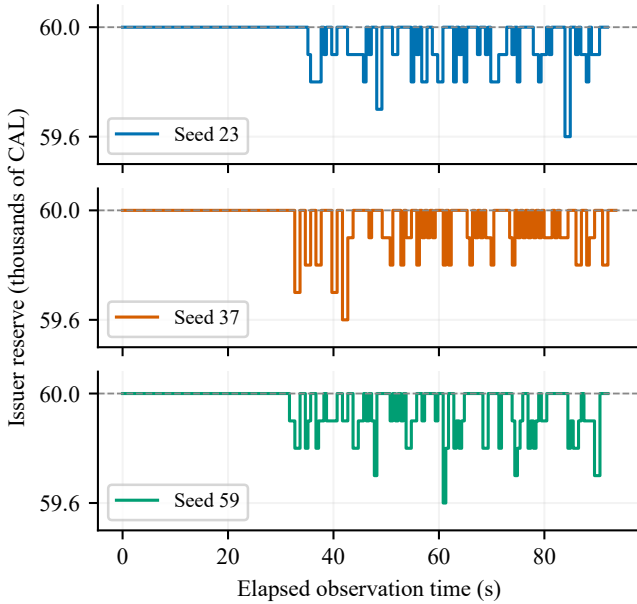


Fig. 6. Sampled reserve trajectories for the three 5% runs (60 compensated sources each). Recovery restores the initial 60,000 CAL. The vertical axis is restricted to 59,600–60,000 CAL to show temporary use. Samples reflect the driver’s release schedule.

this configuration; these pairs do not isolate the responsible stage.

Storage control. Six runs of 1,000 payments at 100 payments/s on the nine-service configuration (committee on disk, wallets NoSync) differ only in the member store. With NoSync members, receipt P50 is 1.002–1.035 ms and P95 33.97–40.62 ms, with 93.11–93.93 payments/s completed including drain. With synchronous member commits, P50 is 43.19–51.85 ms and P95 80.98–96.56 ms, with 90.99–92.72 payments/s. All payments complete. This isolates the cost of synchronous member commits; wallets remain NoSync, so this is a member-commit control under the execution model of Section III.

Real external capital sustains longer operation in the earlier-build capital series, in which no compensation occurred.

TABLE III

ACTUAL CAPITAL ADDITIONS. EACH CONDITION HAS ONE 300-S RUN, STARTS WITH 3,600 CAL, AND COMPLETES 12,000 PAYMENTS. BOTH COLUMNS ARE CAL; EVERY ADDITION DEBITS AN EXTERNAL FUNDED ACCOUNT.

Workload	Final reserve	Added capital
Constant: 20 units/s	14,400	10,800
Step: 10 to 30 units/s	21,600	18,000
Parent delayed by 3 s	23,300	19,700

Separate 300-s experiments use one Worker per member and ample user FUEL, with one run per condition. Starting at 3,600 CAL, constant-rate, rate-step, and 3-s delayed-parent workloads each complete 6,000 two-hop units through the transfers in Table III. The controller uses the experiment’s declared offered flow. Corresponding fixed-high-reserve controls complete; a fixed 3,600-CAL control closes 46 of 6,000 units and retains partial approvals. The supplement gives the controller and all seven conditions, including their admission limits.

C. Sharing Repair Work While Preserving Successors

Same-block batching shares representation work without merging the identities of the compensated obligations. Structural microbenchmarks on Windows amd64 with Go 1.27.1 place $k \in \{1, 2, 4, 8\}$ missing inputs in one historical block and part. Across five repeats, single repairs require k commands and k part adaptations, whereas a batch uses one of each; input adaptations remain k . At $k = 8$, total encoded command bytes fall from 127,336 to 2,752 (Fig. 7).

The dispersed control separates batching from compact encoding. Eight independent target blocks still require eight commands and eight part adaptations, while bytes fall from 30,008 to 5,440. At $k = 1$, both formats perform one part adaptation and encode 3,751 versus 680 bytes. Thus, compact fields explain part of the byte reduction, while shared block representation explains the reduced adaptation count. These are encoded command sizes, not total network traffic or total cryptographic work.

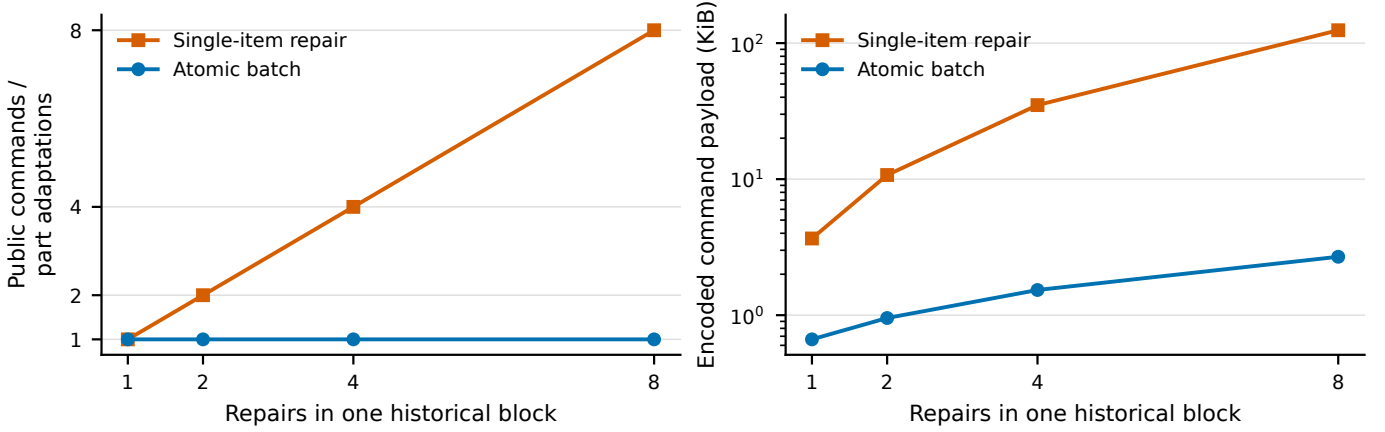


Fig. 7. Same-block, same-part repair costs versus obligation count k . Panels show command/part-adaptation counts and total encoded command payloads in KiB (log scale). Five repeats per configuration give identical structural measurements. Input adaptations remain k .

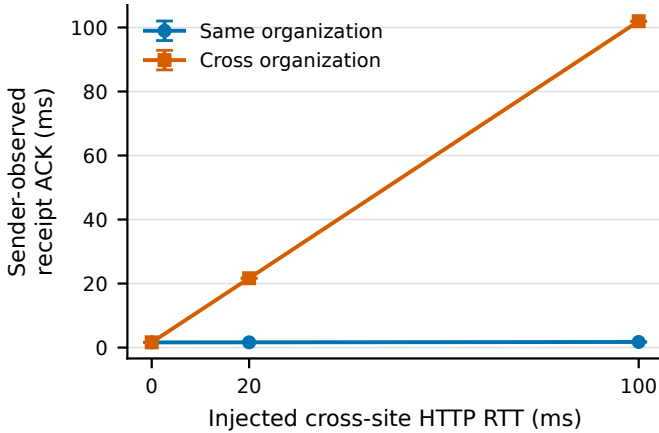


Fig. 8. ACK latency (ms) versus injected cross-site HTTP RTT (ms). Each point is the median of three run-level P50s for the first 120-s cohort; error bars span those P50s' minimum and maximum. Aggregate offered rate is 100 payments/s. Committee P2P is undelayed.

Cross-layer regression executions check successor preservation and economic consistency. For a payment using 40/60 inputs and its already executed successor, source repair preserves original authorizations, output identities, and complete block identities. Comparisons at a fixed prestate, time, and item order yield the same economic projection as sequential compensation. Replaying the same original public history reproduces each height's application hash. The regressions also exercise cross-part faults, atomic batch rejection, duplicate commands, and interleaved-target installation.

An archived fourteen-service, disk-backed batch-integration run combines four actual missing sources into a single 1,568-byte batch whose BatchID agrees across all committee replicas. After late-parent handling, all 1,000 subsequent payments offered at 100 payments/s complete. Receipt P50/P95 are 1.048/26.531 ms; total completion time is 11.120 s, including observation and shutdown. Gaps and outboxes drain, and committee state hashes agree. The structural measurements isolate representation costs; this integration run exercises repair and subsequent payment processing across the deployed services.

D. Continuation Across Registered Organizations

Cross-organization continuation follows the same certificate path, with latency reflecting the injected communication distance. On an archived build recorded in the supplement, the two-organization suite completes 342,000 payments over 24 runs. Two wallet processes each hold 32 identities across 64 lanes; each organization supplies 50 payments/s. The latency comparison uses ten-hop same/cross-organization workloads at 0, 20, and 100 ms injected HTTP RTT, three repetitions per condition. Each proxy direction adds half the RTT; intra-organization certification and committee P2P receive no delay injection.

For the common first-120-s cohort, the median of three run-level cross-group ACK P50s is 1.632, 21.658, and 101.967 ms, respectively; same-group values span 1.615–1.744 ms (Fig. 8). ACKs include the return path, but a recipient may initiate its successor upon acceptance without waiting for that ACK to reach the previous sender. At 100-ms cross-group RTT, three 300-s runs each complete 30,000 payments; pending peaks are 106–110 and drain without sustained tail growth. The experiment isolates HTTP path delay within one physical host.

E. Operating Capacity and Service Availability

Current-build independent-input throughput. Three 30,000-payment runs target 2,000 payments/s on the nine-service in-memory configuration with NoSync wallets and authorized organization-paid FUEL. Member processes use a 200% Go GC target and 16 runtime processors. All payments complete and the four committee states agree. Completion rates including drain are 1,750.27, 1,820.91, and 1,887.31 payments/s over 15.9–17.1 s, with receipt P50s of 2.646, 2.842, and 2.824 ms and P95s of 47.9, 49.1, and 48.0 ms. The driver recorded 6,551–7,570 payments that had to wait for a fast-receipt admission slot and the P95 dispatch lag was 179–229 ms, so the offered load was not uniform at 2,000 payments/s. These are short-run completion measurements; the supplement separately identifies the build and duration of the 150,000-payment series.

On archived builds, redundant certification and installed evidence sustain service under the specified failures. Nine runs at 200 payments/s complete 216,000 payments with one member delayed or paused for 30 s. Gateway-loss controls with one complete installed copy show backup submission 2.503–2.508 s after INSTALL; progress requires a surviving complete copy. Conflict controls produce neither dual conflicting QCs nor repeated principal/fee effects, while partial approvals remain a capacity cost.

The supplement retains frozen-build mappings and complete per-run records, including multi-identity CPU/HTTP costs and historical memory growth separately from backlog. Its 2,048 identities share clients rather than representing 2,048 processes. It also documents a separately configured LND deployment reference with its own completion endpoint and persistence settings.

VII. CONCLUSION

Next supports continuous UTXO payments by separating certified receipt from public execution and source resolution. Its layered architecture lets recipients request successors while evidence replication proceeds in the background. Direct issuer obligations permit missing-source execution with real reserve backing; cumulative accounting covers hidden certificates, and late-source recovery repays the issuing reserve. For completed source sets, delay preserves the final allocation of principal. Authorized repair then preserves fixed payment fields and complete block identities while batching representation work and allowing installation to advance through committed revisions. Original-history replay remains independent of installation timing. The safety analysis connects these mechanisms to unique consumption, funded liabilities, and asset conservation. Dependent-payment measurements and disk-backed repair executions demonstrate the implemented paths. Within the registered-organization model, Next makes recipient continuation the foreground operation and source resolution a funded, auditable background responsibility.

ACKNOWLEDGMENTS

ChatGPT and Claude assisted with drafting and revising the abstract, main-text sections, and Chinese reading edition. Codex assisted with source-grounded review, revision, LaTeX assembly, and plotting scripts. Experimental plots were generated from recorded measurements.

REFERENCES

- [1] M. Baudet, G. Danezis, and A. Sonnino, “FastPay: High-Performance Byzantine Fault Tolerant Settlement,” in *Proceedings of the ACM Conference on Advances in Financial Technologies*, 2020, pp. 163–177.
- [2] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, “Zef: Low-latency, Scalable, Private Payments,” in *Proceedings of the Workshop on Privacy in the Electronic Society*, 2023, pp. 1–16.
- [3] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, “Sui Lutris: A Blockchain Combining Broadcast and Consensus,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2606–2620.
- [4] L. Aumayr, P. Moreno-Sanchez, A. Kate, and M. Maffei, “Blitz: Secure Multi-Hop Payments Without Two-Phase Commits,” in *30th USENIX Security Symposium*, 2021, pp. 4043–4060. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/aumayr>

- [5] —, “Breaking and Fixing Virtual Channels: Domino Attack and Donner,” in *Network and Distributed System Security Symposium*, 2023.
- [6] S. Sridhar, A. Sonnino, and L. Kokoris-Kogias, “Stingray: Fast Concurrent Transactions Without Consensus,” 2025, arXiv:2501.06531, version 1. [Online]. Available: <https://arxiv.org/abs/2501.06531>
- [7] S. Mazumdar and S. Ruj, “CryptoMaze: Privacy-Preserving Splitting of Off-Chain Payments,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 2, pp. 1060–1073, 2023.
- [8] V. Mavroudis, K. Wüst, A. Dhar, K. Kostianen, and S. Čapkun, “Snappy: Fast On-Chain Payments with Practical Collaterals,” in *Network and Distributed System Security Symposium*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.01278>
- [9] A. Madhusudan, M. Sedaghat, S. Tiwari, K. Cong, and B. Preneel, “Reusable, instant and private payment guarantees for cryptocurrencies,” in *Information Security and Privacy (ACISP)*, 2023, pp. 580–605.
- [10] Z. Liu, Y. Xiang, J. Shi, P. Gao, H. Wang, X. Xiao, B. Wen, Q. Li, and Y.-C. Hu, “Make Web3.0 Connected,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 2965–2981, 2022.
- [11] G. Ateniese, B. Magri, D. Venturi, and E. Andrade, “Redactable Blockchain—or—Rewriting History in Bitcoin and Friends,” in *IEEE European Symposium on Security and Privacy*, 2017, pp. 111–126. [Online]. Available: <https://eprint.iacr.org/2016/757>
- [12] D. Zhang, J. Le, X. Lei, T. Xiang, and X. Liao, “Secure Redactable Blockchain With Dynamic Support,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 717–731, 2024.
- [13] X. Li, J. Xu, L. Yin, Y. Lu, Q. Tang, and Z. Zhang, “Escaping From Consensus: Instantly Redactable Blockchain Protocols in Permissionless Setting,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 3699–3715, 2023.
- [14] K. Y. Chan, L. Chen, Y. Tian, and T. H. Yuen, “Reconstructing Chameleon Hash: Full Security and the Multi-Party Setting,” in *Proceedings of the ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1076–1091.
- [15] S. A. K. Thyagarajan, A. Bhat, B. Magri, D. Tschudi, and A. Kate, “Reparo: Publicly Verifiable Layer to Repair Blockchains,” in *Financial Cryptography and Data Security*, 2021, pp. 37–56. [Online]. Available: <https://arxiv.org/abs/2001.00486>
- [16] R. Matzutt, V. Ahlrichs, J. Pennekamp, R. Karwacik, and K. Wehrle, “A Moderation Framework for the Swift and Transparent Removal of Illicit Blockchain Content,” in *IEEE International Conference on Blockchain and Cryptocurrency*, 2022.
- [17] X. Chen and Y. Gao, “CDEdit: A Highly Applicable Redactable Blockchain with Controllable Editing Privilege and Diversified Editing Types,” 2022, arXiv:2205.07054. [Online]. Available: <https://arxiv.org/abs/2205.07054>
- [18] E. Buchman, J. Kwon, and Z. Milosevic, “The latest gossip on BFT consensus,” 2019, arXiv:1807.04938, version 3. [Online]. Available: <https://arxiv.org/abs/1807.04938v3>